KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY (KIIT)

(Deemed to be University)

SCHOOL OF COMPUTER APPLICATIONS

SPRING SEMESTER 2025-26

Date: 11.03.2026

1. **Course Code: BCA3004**
2. **Course title: Introduction to Data Science (IDS) Lab**

Assignment 6 (Dimensionality Reduction)

Title of the assignment:

1. Download mushrooms.csv from Kaggle.com and perform the Principal Component Analysis (PCA) on the dataset.
2. Perform Linear Discriminant Analysis (LDA) on IRIS dataset.

1. Feature selection and extraction are one of the most important steps that must be performed in order for machine learning to be successful. Among the various techniques the Principal Component Analysis (PCA) most frequently used, and the implementation of PCA is given below:

```
# PRINCIPAL COMPONENT ANALYSISI(PCA)
# Importing the libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
import warnings
warnings.filterwarnings("ignore")
m_data = pd.read_csv('mushrooms.csv')

# Machine learning systems work with integers, we need to encode these
# string characters into ints
encoder = LabelEncoder()
# Now apply the transformation to all the columns:
for col in m_data.columns:
 m_data[col] = encoder.fit_transform(m_data[col])
X_features = m_data.iloc[:,1:23]
y_label = m_data.iloc[:, 0]
# Scale the features
scaler = StandardScaler()
X_features = scaler.fit_transform(X_features)
# Visualize
```

```
# Visualize
pca = PCA()
pca.fit_transform(X_features)
pca_variance = pca.explained_variance_
plt.figure(figsize=(8, 6))
plt.bar(range(22), pca_variance, alpha=0.5, align='center',  label='individual variance')
plt.legend()
plt.ylabel('Variance ratio')
plt.xlabel('Principal components')
plt.show()
pca2 = PCA(n_components=17)
pca2.fit(X_features)
x_3d = pca2.transform(X_features)
plt.figure(figsize=(8,6))
plt.scatter(x_3d[:,0], x_3d[:,5], c=m_data['class'])
plt.show()
```

2. Linear discriminant analysis is a method we can use when we have a set of predictor variables and we would like to classify a response variable into two or more classes.

Step 1: Load Necessary Libraries

First, we'll load the necessary functions and libraries for this example:

```
from sklearn.model_selection import train_test_split
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.model_selection import cross_val_score
```

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn import datasets
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

Step 2: Load the Data

For this example, we'll use the iris dataset from the sklearn library. The following code shows how to load this dataset and convert it to a pandas DataFrame to make it easy to work with:

```
#load iris dataset
iris = datasets.load_iris()

#convert dataset to pandas DataFrame
df = pd.DataFrame(data = np.c_[iris['data'], iris['target']],
                  columns = iris['feature_names'] + ['target'])
df['species'] = pd.Categorical.from_codes(iris.target, iris.target_names)
df.columns = ['s_length', 's_width', 'p_length', 'p_width', 'target', 'species']

#view first six rows of DataFrame
df.head()
```

We can see that the dataset contains 150 total observations.

For this example, we'll build a linear discriminant analysis model to classify which species a given flower belongs to.

We'll use the following predictor variables in the model:

- Sepal length
- Sepal width
- Petal length
- Petal width

And we'll use them to predict the response variable Species, which takes on the following three potential classes:

- Setosa
- versicolor
- virginica

Step 3: Fit the LDA Model

Next, we'll fit the LDA model to our data using the LinearDiscriminantAnalsyis function from sklearn:

```
#define predictor and response variables
X = df[['s_length', 's_width', 'p_length', 'p_width']]
y = df['species']
```

```
#Fit the LDA model
model = LinearDiscriminantAnalysis()
model.fit(X, y)
```

Step 4: Use the Model to Make Predictions

Once we've fit the model using our data, we can evaluate how well the model performed by using repeated stratified k-fold cross validation.

For this example, we'll use 10 folds and 3 repeats:

```
#Define method to evaluate model
cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=1)

#evaluate model
scores = cross_val_score(model, X, y, scoring='accuracy', cv=cv, n_jobs=-1)
print(np.mean(scores))
```

We can see that the model performed a mean accuracy of 97.78%.

We can also use the model to predict which class a new flower belongs to, based on input values:

```
#define new observation
new = [5, 3, 1, .4]

#predict which class the new observation belongs to
model.predict([new])

array(['setosa'], dtype='<U10')
```

We can see that the model predicts this new observation to belong to the species called *setosa*.

Step 5: Visualize the Results

Lastly, we can create an LDA plot to view the linear discriminants of the model and visualize how well it separated the three different species in our dataset:

```
#define data to plot
X = iris.data
y = iris.target
model = LinearDiscriminantAnalysis()
data_plot = model.fit(X, y).transform(X)
target_names = iris.target_names

#create LDA plot
plt.figure()
colors = ['red', 'green', 'blue']
lw = 2
for color, i, target_name in zip(colors, [0, 1, 2], target_names):
```

```
    plt.scatter(data_plot[y == i, 0], data_plot[y == i, 1], alpha=.8, color=color,
                label=target_name)

#add legend to plot
plt.legend(loc='best', shadow=False, scatterpoints=1)

#display LDA plot
plt.show()
```

Viva Questions:

1. **Write the differences between Principal Component Analysis (PCA) and Linear discriminant analysis.**

More Information about "**from sklearn import datasets**"

https://scikit-learn.org/stable/datasets.html

In addition, there are also miscellaneous tools to load datasets of other formats or from other locations, described in the Loading other datasets section.